Uniwersytet Łódzki
Wydział Matematyki i Informatyki

Temat ćwiczenia:

Wielowarstwowy perceptron

Sprawozdanie z przedmiotu: ***Zaawansowane Metody Eksploracji***
Studia stacjonarne 2-go stopnia
Kierunek: *Analiza Danych, sem. II*
Prowadzący: *prof. dr hab. inż. Grzegorz Dudek*

Data wykonania ćwiczenia:	7.05.2025
Data złożenia sprawozdania:	21.05.2025
Data zaliczenia:	

Wykonali:	Ocena:
Dawid Sporysiak	

Nr sekcji: **3.19**

A. Cel ćwiczenia

Celem ćwiczenia jest nauka korzystania i testowania sieci neuronowych w praktycznym zastosowaniu na przykładzie danych dotyczących cementu i jego wytrzymałości.

Opis używanej sieci neuronowej

Użyty w tym sprawozdaniu został wielowarstwowy perceptron (MLP, Multilayer Perceptron). Ta sieć należy do neuronowego modelu uczenia nadzorowanego i została użyta do rozwiązania problemu regresji. Problem postawiony w zadaniu był problemem regresyjnym tzn. chcieliśmy znaleźć odpowiednie parametry i jak najmniejsze MSE, natomiast problem klasyfikacyjny dla tego musiałby wyglądać między innymi tak, że dzielimy pewne przedziały wartości na klasy, dla których chcemy zakładać, że model jest dobry, i zbadać procent poprawnych klasyfikacji w oparciu o wyniki sieci.
Zastosowana sieć składa się z następujących części:
- warstwa wejściowa, czyli liczba neuronów równa liczbie cech (8).
- warstw ukrytych (np. (100,50) oznaczające, że sieć jest dwuwarstwowa, i składa się ze 100 oraz 50 neuronów)
- warstwa wyjściowa z 1 neuronem (regresja) lub N neuronami (klasyfikacja N klas). W zadaniu użyto 1 neuronu, aby przewidywać ciągłą wartość csMPa. Zastosowano również funkcje aktywacji, ReLU oraz Tanh wykorzystujące wbudowane funkcje.

Wykorzystana również jest MSE do znalezienia optymalnych parametrów ,parametr alpha z przedziału [0,1] kontrolujący regularyzację L2 (karanie dużych wag, zapobieganie przeuczeniu), oraz 5-krotna crosswalidacja w GridSearchCV do doboru hiperparametrów. Domyślnym optymalizatorem w tym przypadku jest 'Adam' adaptive moment estimation.

B. Treść zadania

Zadanie 1

Treść:

1. Zaimportuj niezbędne moduły.

Rozwiązanie zadania:

```
#zadanie 1
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
#optymalizacja hiperparametrów
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
#wielowarstwowy perceptron do regresji
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import Pipeline
```

Zadanie 2

Treść:

2. Wczytaj dane, dokonaj selekcji danych – każdy student tworzy inny zbiór danych, który składa się z 1000 wylosowanych wierszy danych oryginalnych (skorzystaj z kodu zamieszczonego w ćwiczeniu dot. drzew decyzyjnych).

Rozwiązanie zadania:

```
#Zadanie 2
nr_gr = 3.19
r_k = 2025
data = pd.read_csv("Concrete_Data_Yeh.csv")
#wybór 1000 danych
np.random.seed(int(nr_gr * r_k))
#wylosowanie 1000 wierszy i podmiana indeksów
random_indices = np.random.permutation(len(data))[:1000]
data = data.iloc[random_indices].reset_index(drop=True)

print(f"Liczba danych: {len(data)}")
data.head()
```

Wynik:

Liczba danych: 1000

	cement	slag	flyash	water	superplasticizer	coarseaggregate	fineaggregate	age	csMPa
0	203.5	135.7	0.0	185.7	0.0	1076.2	759.3	7	11.96
1	212.5	0.0	100.4	159.3	8.7	1007.8	903.6	100	42.92
2	200.0	0.0	0.0	180.0	0.0	1125.0	845.0	28	12.25
3	342.0	38.0	0.0	228.0	0.0	932.0	670.0	180	52.12
4	425.0	106.3	0.0	153.5	16.5	852.1	887.1	3	33.40

Wniosek:

Wylosowane zostało 1000 wierszy, następnie poprzepisywane indeksy na nowe z przedziału [0-999].

Zadanie 3

Treść:

3. Sprawdź czy w danych są zduplikowane wiersze i brakujące wartości. Jeśli tak, usuń te wiersze (`data = data.drop_duplicates()`).

Rozwiązanie zadania:

```
#Zadanie 3
#sprawdzanie duplikatów i danych z brakującymi wartościami
duplicates = data.duplicated().sum()
missing_values = data.isnull().sum().sum()

print(f"Liczba duplikatów: {duplicates}")
print(f"Liczba brakujących wartości: {missing_values}\n")

#usuwanie
initial_count = len(data)
```

```
data = data.drop_duplicates().dropna()
removed_count = initial_count - len(data)
print(f"Usunięte {removed_count} duplikaty")
print(f"Usunięto: {1000 - len(data)} wierszy\n")
print(f"Pozostało {len(data)} wierszy")


data.head()
```

Wynik:

```
Liczba duplikatów: 0
Liczba brakujących wartości: 0

Usunięte 0 duplikaty
Usunięto: 24 wierszy

Pozostało 976 wierszy
```

	cement	slag	flyash	water	superplasticizer	coarseaggregate	fineaggregate	age	csMPa
0	203.5	135.7	0.0	185.7	0.0	1076.2	759.3	7	11.96
1	212.5	0.0	100.4	159.3	8.7	1007.8	903.6	100	42.92
2	200.0	0.0	0.0	180.0	0.0	1125.0	845.0	28	12.25
3	342.0	38.0	0.0	228.0	0.0	932.0	670.0	180	52.12
4	425.0	106.3	0.0	153.5	16.5	852.1	887.1	3	33.40

Wniosek:

Sprawdzane było, czy wszystkie wartości w wierszach były powtórzone, oraz czy brakuje wartości w danych, duplikaty nie zostały znalezione w przeciwieństwie do niepełnych danych.

Zadanie 4

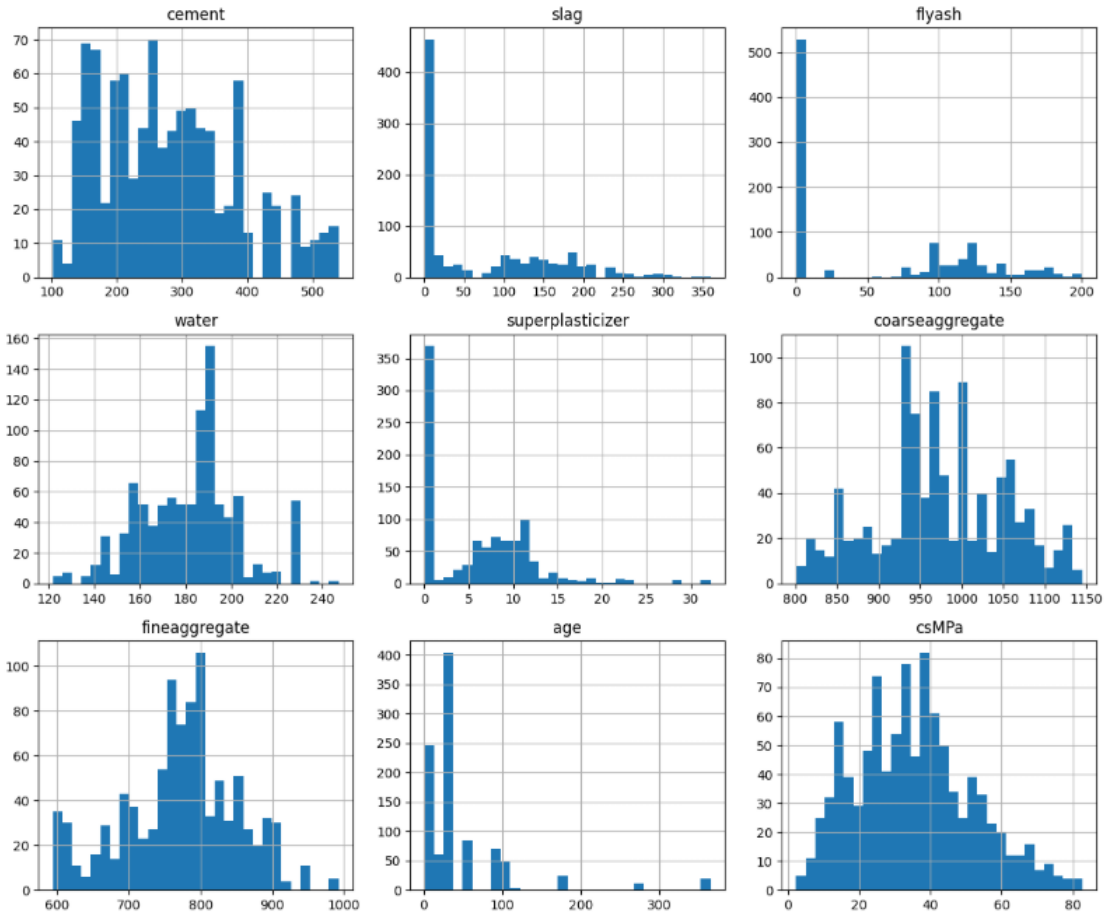
Treść:

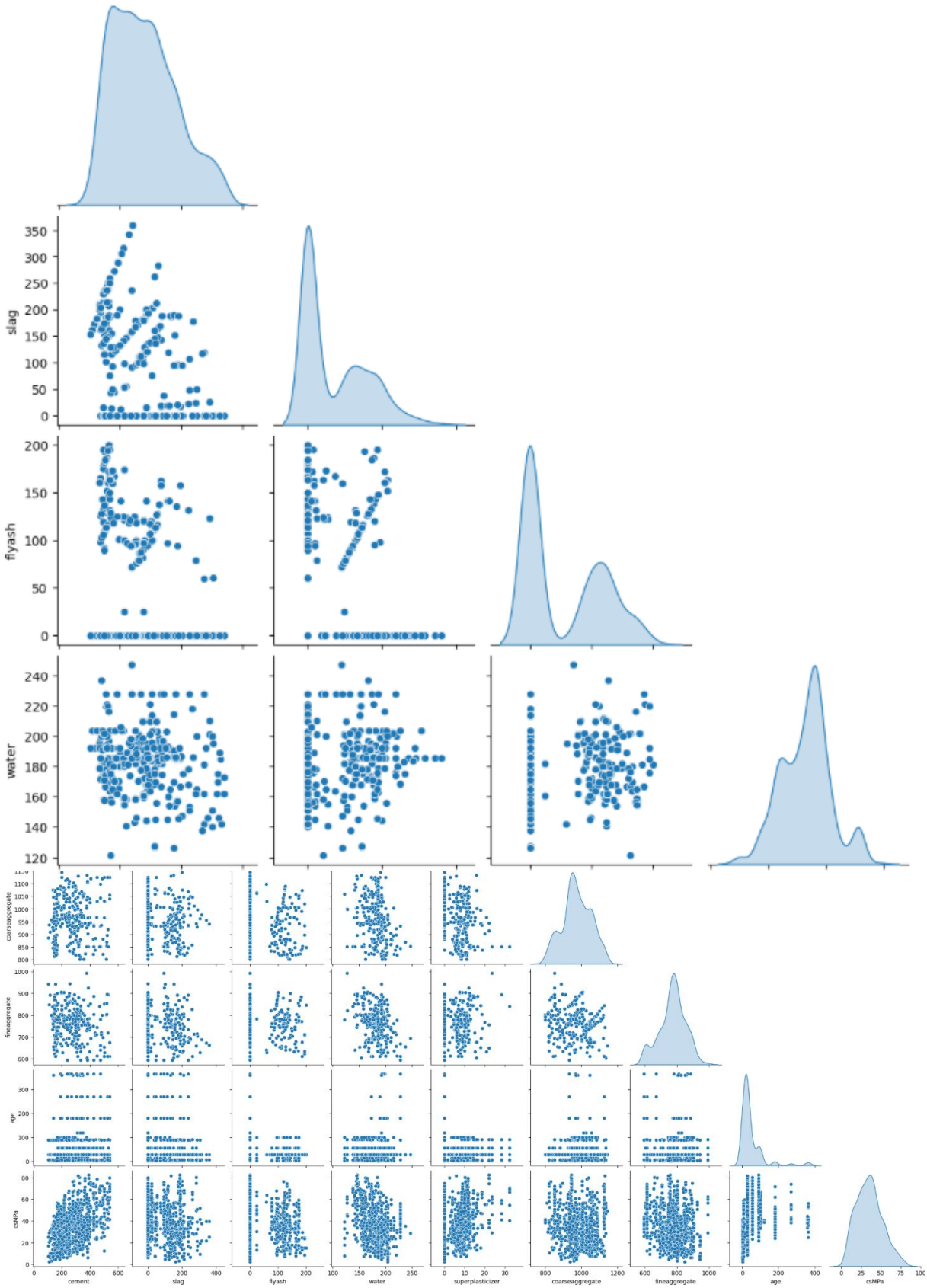
4. Scharakteryzuj dane, (przydatne funkcje: `data.describe()`, `data.info()`), zobrazuj powiązania pomiędzy zmiennymi i ich rozkłady (`sns.pairplot()`) oraz korelacje pomiędzy zmiennymi.

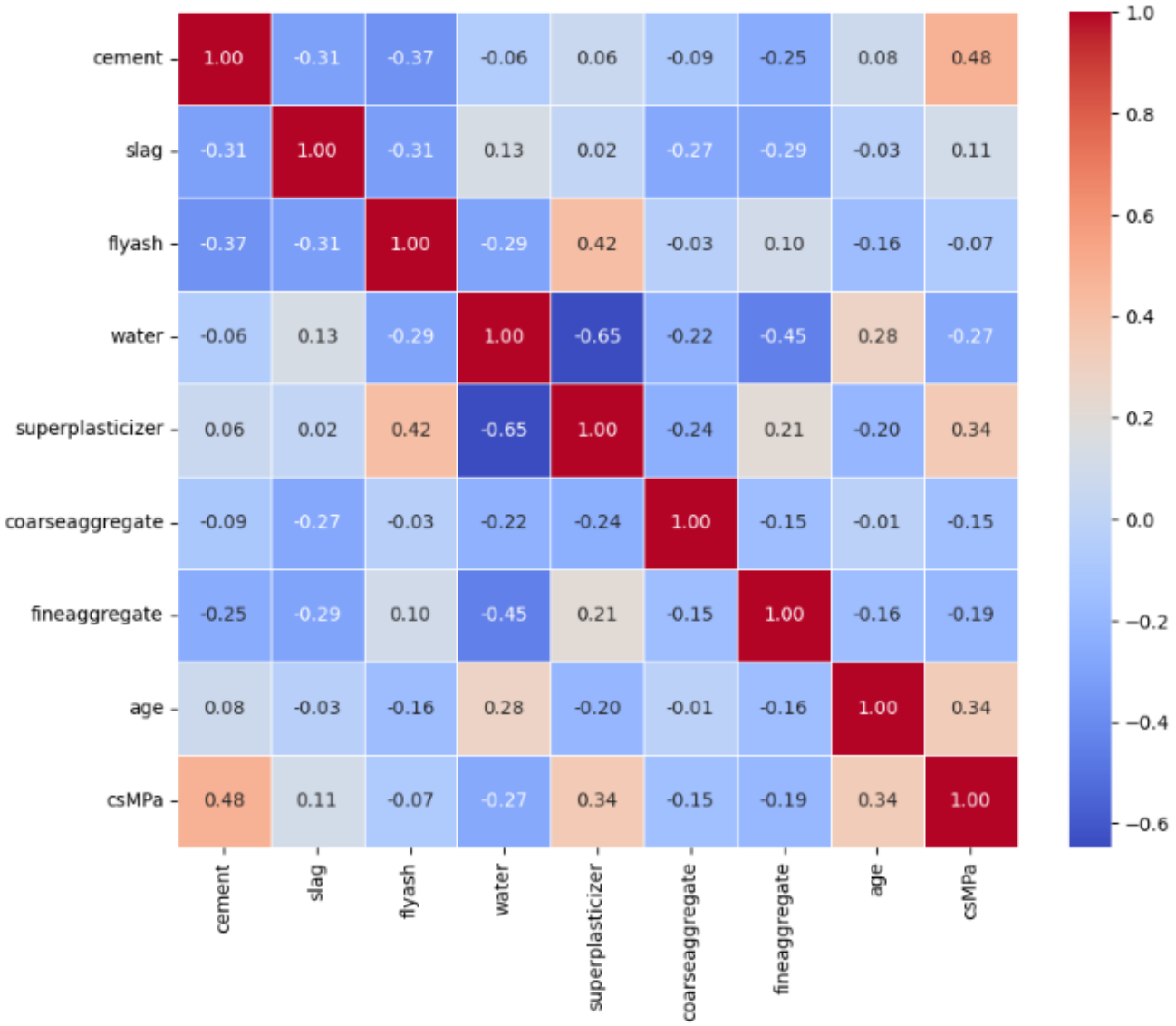
Rozwiązanie zadania:

```
#zadanie 4
#histogramy dla wszystkich zmiennych
print("\nHistogramy zmiennych:")
data.hist(bins=30, figsize=(12, 10), layout=(3, 3))
plt.tight_layout()
plt.show()

#wykresy zależności
print("\nMacierz wykresów rozrzutu:")
sns.pairplot(data, diag_kind='kde', corner=True)
plt.suptitle("Wykresy zależności między zmiennymi", y=1.02)
plt.show()

#heatmapa
plt.figure(figsize=(10, 8))
corr_matrix = data.corr(numeric_only=True)
sns.heatmap(corr_matrix,annot=True,fmt=".2f",cmap='coolwarm',linewidths
=0.5)
```

Wynik:




slag
flyash
water
coarseaggregate
fineaggregate
age
csMPa
cement
slag
flyash
water
superplasticizer
coarseaggregate
fineaggregate
age
csMPa



Wniosek:

Odczytać można z heatmapy, że w największej korelacji z csMPa jest cecha "cement", a następnie "superplasticizer" i "age". Są to korelacje parami, tzn w rzeczywistości niekoniecznie tylko jedna cecha może wpływać na wartość csMPa, a np kilka, ponieważ pojedynczo mogą one dawać niski współczynnik korelacji, ale ich zestaw już może być bardziej obiecujący. Najgorzej wypadły cechy "slag" oraz "flyash".

Zadanie 5

Treść:

5. Przygotuj dane do uczenia modelu. Utwórz tabele:
   - `X` z przykładami (zawierającą wszystkie kolumny z wyjątkiem `'csMPa'`) i
   - `y` z etykietami (kolumna `'csMPa'`).

   Podziel dane na część treningową (80% danych) i testową (20%).
   Dokonaj standaryzacji atrybutów w zbiorach treningowym i testowym.

Rozwiązanie zadania:

```
#Zadanie 5
X = data.drop('csMPa', axis=1)
#X =  pozostałe cechy
y = data['csMPa']
#Y = tylko wytrzymałość
print(X.head())
print('\n')
print( y.head())
print('\n')
#podział danych
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1)

#standaryzacja
scaler = StandardScaler()
X_treningowe_stand = scaler.fit_transform(X_train)
X_testowe_stand = scaler.transform(X_test)

X_treningowe_stand = pd.DataFrame(X_treningowe_stand,
columns=X.columns)
X_testowe_stand = pd.DataFrame(X_testowe_stand, columns=X.columns)
print(X_treningowe_stand.head())
```

Wynik:

```
   cement   slag  flyash  water  superplasticizer  coarseaggregate  \
0   203.5  135.7     0.0  185.7               0.0           1076.2
1   212.5    0.0   100.4  159.3               8.7           1007.8
2   200.0    0.0     0.0  180.0               0.0           1125.0
3   342.0   38.0     0.0  228.0               0.0            932.0
4   425.0  106.3     0.0  153.5              16.5            852.1

   fineaggregate  age
0          759.3    7
1          903.6  100
2          845.0   28
3          670.0  180
4          887.1    3


0    11.96
1    42.92
2    12.25
3    52.12
4    33.40
Name: csMPa, dtype: float64


     cement      slag    flyash     water  superplasticizer  coarseaggregate
0  0.003999 -0.650333  1.393533 -0.134666         -0.090379        -1.693336
1  0.173533 -0.825320 -0.842930  0.987701         -0.995439         0.008612
2  0.107603 -0.825320 -0.842930  0.145926         -0.995439         1.233499
3  0.188602  0.806723 -0.842930  1.478736          0.024810        -1.003531
4  0.248881 -0.825320 -0.842930  0.473283         -0.995439        -0.042962

   fineaggregate       age
0       0.329278 -0.276053
1       0.525475 -0.276053
2      -0.063115 -0.276053
3      -0.633310 -0.276053
4       0.930130  2.066571
```

Wniosek:

Podzielono dane na X,Y wszystkie cechy poza csMPa, csMPa, a następnie zestandaryzowane((wartość-średnia danej kolumny zmiennych)/odchylenie standardowe kolumny zmiennych) aby uniknąć dużych rozbieżności wartości. Wyświetlone kolejno są:
- dane bez standaryzacji (tylko X)
- dane bez standaryzacji (tylko Y)
- dane po standaryzacji (tylko X)

Zadanie 6

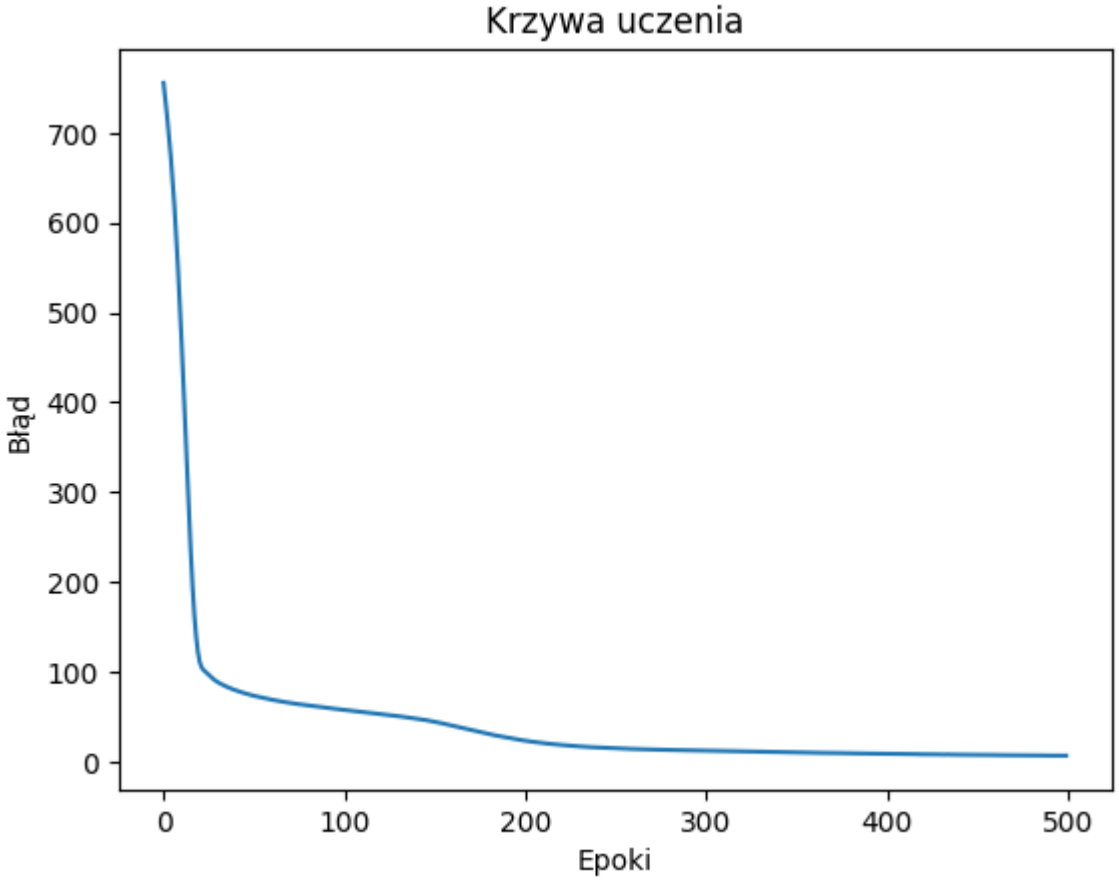
Treść:

Przeprowadź próbne uczenie modelu. Skorzystaj z implementacji sieci neuronowej w Scikit-learn, PyTorch lub TensorFlow + Keras (patrz ostatnia część wykładu). Pokaż krzywą zbieżności (*loss_curve*). Pokaż następujące metryki/wykresy dla danych testowych: MSE, histogram błędów, wykres *Actual value vs. Predicted value*.

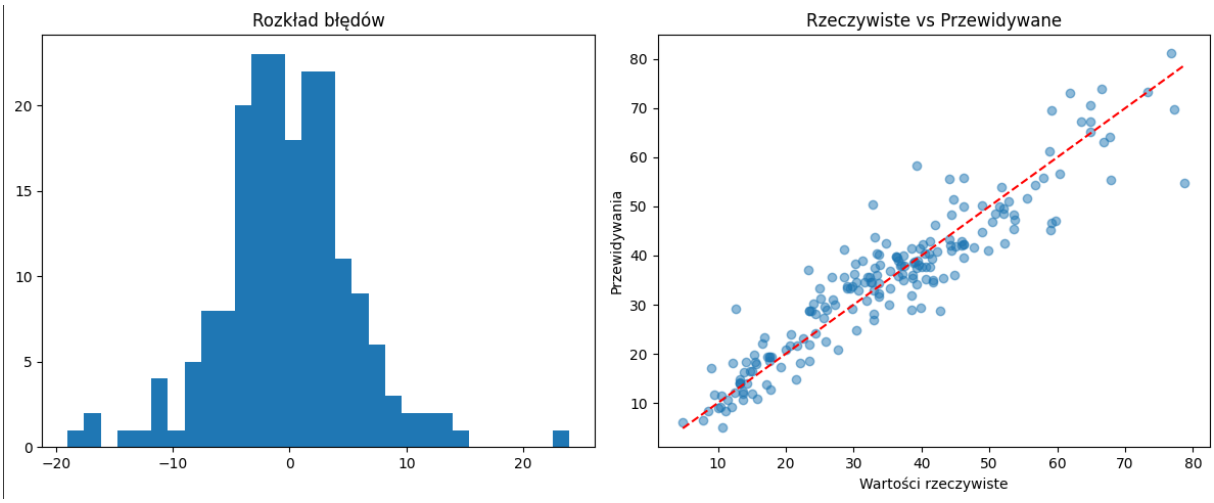
Rozwiązanie zadania:

```
#model sieci neuronowej i trenowanie
model =
MLPRegressor(hidden_layer_sizes=(100,100),activation='relu',max_iter=50
0,random_state=4000)
model.fit(X_treningowe_stand, y_train)

#wykres krzywej uczenia
plt.plot(model.loss_curve_)
plt.title('Krzywa uczenia')
plt.xlabel('Epoki')
plt.ylabel('Błąd')
plt.show()

#testowanie
predictions = model.predict(X_testowe_stand)
mse = mean_squared_error(y_test, predictions)
print(f"Błąd MSE: {mse:.2f}")
plt.figure(figsize=(12, 5))

#rozkład błędów
plt.subplot(1, 2, 1)
errors = y_test - predictions
plt.hist(errors, bins=30)
plt.title('Rozkład błędów')

#krzywa regresji
plt.subplot(1, 2, 2)
plt.scatter(y_test, predictions, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--')
plt.title('Rzeczywiste vs Przewidywane')
plt.xlabel('Wartości rzeczywiste')
plt.ylabel('Przewidywania')
plt.tight_layout()
plt.show()
```

Wynik:




```
Błąd MSE: 32.92
```



Wniosek:

W tym zadaniu zostało zastosowane tworzenie modelu sieci neuronowej na danych treningowych. Z wykresu krzywej uczenia wynik poprawiał się wraz ze wzrostem liczby epok. Rozkład błędów przypomina wyglądem rozkład normalny. Dla podanych parametrów sieci otrzymaliśmy MSE=32.92, przy czym z pewnością nie jest to najoptymalniejszy wynik.

Zadanie 7

Treść:

Zoptymalizuj hiperparametry w procedurze *Grid Search*. Zobrazuj wyniki optymalnego modelu podobnie jak w p. 6.

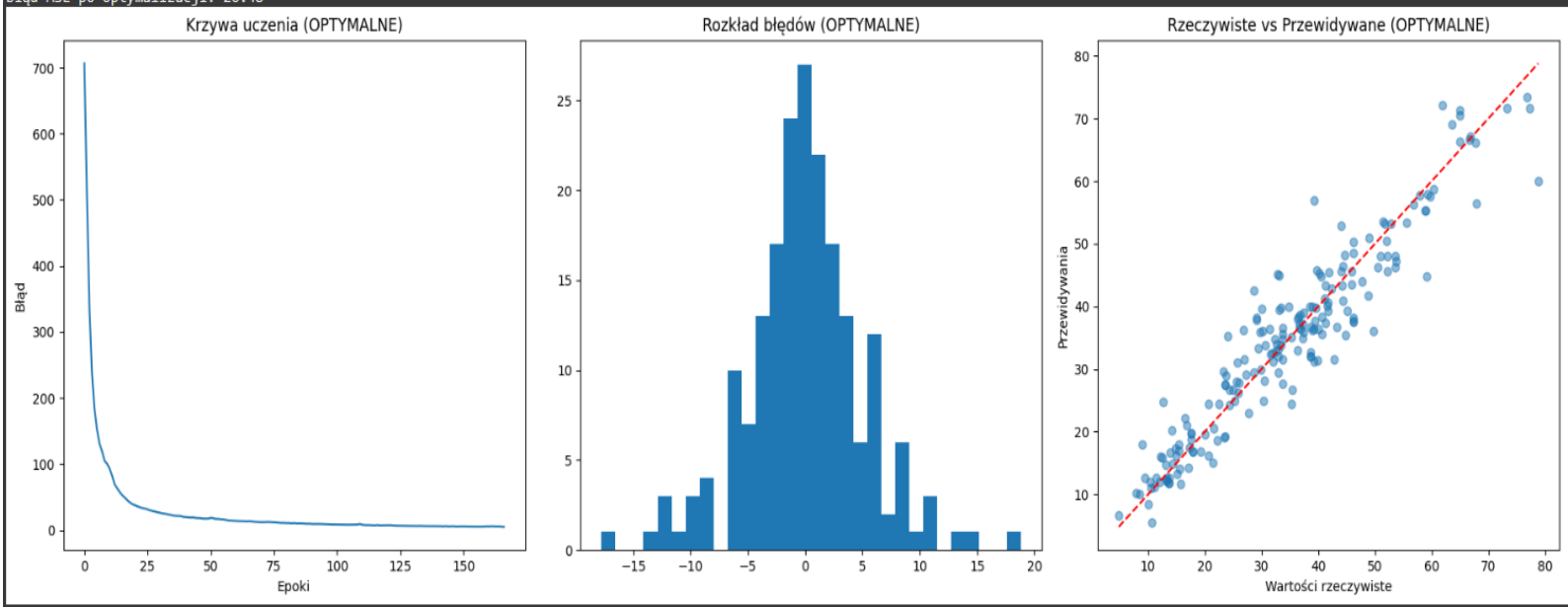
Rozwiązanie zadania:

```
#zadanie 7
#siatka parametrów
param_grid = {'hidden_layer_sizes': [(100), (100, 50), (100,
100)],'activation': ['relu', 'tanh'],'alpha': [0.0001, 0.001, 0.01]
'learning_rate_init': [0.01, 0.5, 0.1],'max_iter': [500, 700]}

#GridSearch
model = MLPRegressor(random_state=4000, early_stopping=True)
grid_search =
GridSearchCV(estimator=model,param_grid=param_grid,cv=5,scoring='neg_me
an_squared_error')
grid_search.fit(X_treningowe_stand, y_train)
#najlepszy wynik
print("Najlepsze parametry:", grid_search.best_params_)
best_model = grid_search.best_estimator_

predictions = best_model.predict(X_testowe_stand)
mse = mean_squared_error(y_test, predictions)
print(f"Błąd MSE po optymalizacji: {mse:.2f}")

plt.figure(figsize=(18, 6))
```

```
#krzywa uczenia
plt.subplot(1, 3, 1)
plt.plot(best_model.loss_curve_)
plt.title('Krzywa uczenia (OPTYMALNE)')
plt.xlabel('Epoki')
plt.ylabel('Błąd')

#rozkład błędów
plt.subplot(1, 3, 2)
errors = y_test - predictions
plt.hist(errors, bins=30)
plt.title('Rozkład błędów (OPTYMALNE)')

#prosta regresji
plt.subplot(1, 3, 3)
plt.scatter(y_test, predictions, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--')
plt.title('Rzeczywiste vs Przewidywane (OPTYMALNE)')
plt.xlabel('Wartości rzeczywiste')
plt.ylabel('Przewidywania')

plt.tight_layout()
plt.show()
print(f"Poprawa MSE: {((32.92 - mse)/32.92 * 100):.2f}%")
```

Wynik:

```
Fitting 5 folds for each of 108 candidates, totalling 540 fits
Najlepsze parametry: {'activation': 'tanh', 'alpha': 0.01, 'hidden_layer_sizes': (100, 100), 'learning_rate_init': 0.01, 'max_iter': 500}
Błąd MSE po optymalizacji: 26.48
```




```
Poprawa MSE: 18.52%
```

Wniosek:

W tym zadaniu główną częścią jest zdefiniowanie siatki parametrów oraz funkcji grid_search. Grid_search przemnożył wszystkie możliwe kombinacje z podanych wartości rozmiarów i warstw sieci, funkcji aktywacji, wartości parametru alpha, learning rate, oraz max_iter, a następnie powstały iloczyn (liczba możliwych kombinacji) przeszedł 5-krotną crosswalidację, dając łącznie 3*2*3*3*2*5= 540 dopasowań. Wartości jak i ich ilość można zmieniać, ale przy większej ilości kombinacji czas potrzebny na znalezienie optymalnych parametrów jest zbyt duży (dla 540 dopasowań czas wykonania wyniósł około 5 minut). Przy podanych parametrach udało się uzyskać lepszy wynik MSE, mniejszy o 13,1%, ale nie musiało tak być, ponieważ model z zadania 6 mógł okazać się lepszy niż 540 rozpatrzonych przypadków.

Zadanie 8

Treść:

Wybierz jeden z atrybutów, który Twoim zdaniem pozwoli najlepiej przewidzieć wartość zmiennej wyjściowej. Utwórz model sieci neuronowej aproksymującej zmienną wyjściową na podstawie tego atrybutu. Porównaj wyniki p. 8 i 7.

Rozwiązanie zadania:

```
#wybór 'najlepszej' cechy
best_feature = 'cement'
X_single = data[[best_feature]]
y_single = data['csMPa']

#podział i standaryzacja
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_single,
y_single, test_size=0.2)
scaler = StandardScaler()
X_train_s_std = scaler.fit_transform(X_train_s)
X_test_s_std = scaler.transform(X_test_s)

#budowa i trenowanie modelu
single_model =
MLPRegressor(hidden_layer_sizes=(50),activation='relu',max_iter=500)
single_model.fit(X_train_s_std, y_train_s)

#MSE
predictions_single = single_model.predict(X_test_s_std)
mse_single = mean_squared_error(y_test_s, predictions_single)
print(f"MSE dla modelu z jednym atrybutem: {mse_single:.2f}")
print(f"Poprawa: {((mse - mse_single)/mse*100):.2f}%")
```

Wynik:

```
MSE dla modelu z jednym atrybutem: 216.84
Poprawa: -667.74%
```

Wniosek:

Otrzymany wynik sugeruje, że branie pod uwagę tylko jednej cechy przy badaniu wartości csMPa jest nieodpowiednie, tzn wartość MSE wzrosła ponad 6-krotnie. Sugeruje to, że wartość csMPa jest składową kilku cech, przy czym jest to dużo bardziej złożony przypadek niż branie pojedynczej cechy. Być może nawet cechy "slag" oraz "flyash", które w zadaniu 4 wwykazywały najmniejszą korelację z csMPa mogą mieć decydujące znaczenie, ponieważ pomimo braku bezpośredniego wpływu na csMPa mogą one mieć dużo większy wpływ na składowe, które z kolei już taki wpływ mają.

Wnioski końcowe:

W powyższym sprawozdaniu zastosowane było filtrowanie, dzielenie, i testowanie danych w sieci neuronowej przy wykorzystaniu biblioteki sklearn oraz regresyjnej sieci neuronowej (MLP). Ważnym wnioskiem jest to, że sama macierz korelacji niekoniecznie dostarcza odpowiednich informacji, ponieważ porównując zadanie 4 i 8 widać, że wykorzystywanie jednej zmiennej z całego zestawu nie musi dawać pozytywnych rezultatów. Wynik był dużo dużo gorszy niż przy wykorzystaniu ich wszystkich. Ważna również jest sama sieć neuronowa zastosowana w zadaniu 6 oraz 7. W zadaniu 6, czyli przy jednej sieci, wynik był nienajgorszy, natomiast w zadaniu 7 przy aż 540 powstałych kombinacjach wynik poprawił się o 18%. Gdyby dodać więcej parametrów, to z pewnością znaleziona by została lepsza kombinacja parametrów, ale czas na jej znalezienie byłby zdecydowanie za duży.